

Experiment No. 12

Pointer in C + Memory dynamic Allocation -II

Objective:

The objectives of this lab are:

1. To understand pointer expressions and pointer arithmetic in C.
2. To explore the relationship between pointers and arrays.
3. To learn the concept of arrays of pointers.
4. To implement real-world applications using pointers through case studies.
5. To understand pointers to functions and their applications.
6. To apply secure coding practices while using pointers in C.
7. To analyze memory access and prevent pointer-related errors.

Prerequisites

Following are the prerequisites of this lab are:

- Understanding of variables, data types, and memory representation in C.
- Knowledge of how functions work and how arguments are passed by value.
- Basic understanding of arrays and strings, including indexing and storage.
- Using address-of (&) and dereference (*) operators in simple expressions.
- Familiarity with the concept of memory addresses and how they relate to variables.
- Understanding the difference between compile-time memory (stack) and dynamic memory (heap).

Pointer Expressions and Pointer Arithmetic

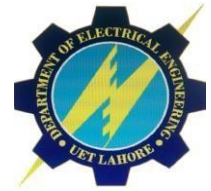
Pointer arithmetic allows operations on pointer variables such as increment (`ptr++`), decrement (`ptr--`), addition, and subtraction. These operations move the pointer based on the size of the data type it points to.

Key Points:

- `ptr + 1` moves the pointer to the next memory location



Programming Fundamentals Lab Manual



- Arithmetic depends on data type size
- Pointer subtraction gives distance between elements.

Example:

```
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};
    int *ptr = arr;

    printf("%d\n", *ptr);
    ptr++;
    printf("%d\n", *ptr);

    return 0;
}
```

Output:

```
10
20
```

Arrays of Pointers

An **array of pointers** is a data structure in which **each element of the array is a pointer**. Instead of storing actual data values, the array stores **memory addresses** of variables, arrays, or strings.

In C, arrays of pointers are widely used when:

- Working with **strings**
- Handling **dynamic data**
- Managing **large data efficiently**
- Avoiding unnecessary copying of data



Programming Fundamentals Lab Manual



Example:

```
#include <stdio.h>

int main() {
    int a = 5, b = 10, c = 15;
    int *arr[3] = {&a, &b, &c};

    for(int i = 0; i < 3; i++)
        { printf("Value = %d\n", *arr[i]);
        }

    return 0;
}
```

Output:

```
Value = 5
Value = 10
Value = 15
```

Case Study: Card Shuffling and Dealing Simulation

This case study demonstrates how **arrays and pointers** can be used together to simulate **card shuffling and dealing** in a card game. A standard deck of cards consists of **52 cards**, divided into **4 suits** (Hearts, Diamonds, Clubs, Spades) and **13 faces** (Ace to King).

Using pointers allows us to **access, shuffle, and deal cards efficiently** without copying card data. Instead of moving card values, **only their positions in memory are accessed**, making the program faster and memory-efficient.

Concept Explanation

- A **2D array** is used to represent the deck of cards.
- Pointers are used to **access card values dynamically**.
- The rand() function is used to **shuffle** cards randomly.
- Cards are **dealt sequentially** after shuffling.



Programming Fundamentals Lab Manual



Deck Representation

Each card is represented by:

- A **row** (suit)
- A **column** (face)

Example:

- `deck[0][0]` → Ace of Hearts
- `deck[3][12]` → King of Spades

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* Function Prototypes */
void shuffle(int deck[4][13]);
void deal(const int deck[4][13]);

int main() {
    int deck[4][13] = {0}; // Deck initialized to 0

    srand(time(NULL));    // Seed random number generator

    shuffle(deck);        // Shuffle the deck
    deal(deck);           // Deal the cards

    return 0;
}

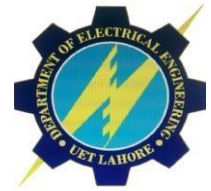
/* Function to shuffle the cards */
void shuffle(int deck[4][13]) {
    int card, row, column;

    for (card = 1; card <= 52; card++)
        { do {
            row = rand() % 4;    // Random suit
            column = rand() % 13; // Random face
        } while (deck[row][column] != 0);

        deck[row][column] = card;
    }
}
```



Programming Fundamentals Lab Manual



```
/* Function to deal the cards */
void deal(const int deck[4][13]) {
    const char *suit[] = {"Hearts", "Diamonds", "Clubs", "Spades"};
    const char *face[] = {
        "Ace", "Two", "Three", "Four", "Five", "Six",
        "Seven", "Eight", "Nine", "Ten", "Jack", "Queen", "King"
    };

    int card, row, column;

    for (card = 1; card <= 52; card++)
        { for (row = 0; row < 4; row++) {
            for (column = 0; column < 13; column++)
                { if (deck[row][column] == card) {
                    printf("%s of %s\n", face[column], suit[row]);
                }
            }
        }
    }
}
```

Output:

```
Four of Diamonds
King of Spades
Ace of Hearts
Ten of Clubs
```

Pointers to Functions

In C, a **function pointer** is a pointer that stores the **address of a function** instead of a variable. Just like pointers can point to data, **function pointers point to executable code**. They enable **dynamic function calls**, meaning the function to be executed can be decided **at runtime**.

Function pointers are especially useful in modular programming, where different functions share the same interface but perform different operations



Programming Fundamentals Lab Manual



Applications of Function Pointers

Callback function

A **callback function** is a function that is **passed as an argument to another function** and is **called inside that function**. The calling function does not know the exact operation in advance; instead, it “calls back” the function passed to it.

Callback functions are commonly used in:

- Sorting functions
- Event handling
- Menu-driven systems
- Library functions

Example:

```
#include <stdio.h>

/* Callback functions */
void sayHello() {
    printf("Hello! This is a callback function.\n");
}

void sayGoodbye() {
    printf("Goodbye! This is another callback function.\n");
}

/* Function that accepts a callback */
void execute(void (*callback)()) {
    printf("Executing callback...\n");
    callback(); // Calling the callback function
}

int main() {
    execute(sayHello); // Passing function as argument
    execute(sayGoodbye); // Passing another function

    return 0;
}
```



Programming Fundamentals Lab Manual



Lab Tasks

Task 1

Write a C program using pointer arithmetic to traverse an integer array and display its elements.

Code

```
#include <stdio.h>

int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // point to the start of the array
    int *ptr = numbers;

    printf("array elements using pointer arithmetic:\n");

    // pointing the pointer forward in memory
    for (int i = 0; i < size; i++) {
        printf("%d ", *ptr);
        ptr++; // move to the next int block
    }
    printf("\n");

    return 0;
}
```

Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ make task1
gcc -c task1.c
gcc -o task1 task1.o
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ ./task1
array elements using pointer arithmetic:
10 20 30 40 50
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ |
```



Programming Fundamentals Lab Manual



Task 2

Write a program to access array elements using pointer notation instead of indexing.

Code:

```
#include <stdio.h>

int main() {
    int arr[] = {5, 15, 25, 35, 45};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("accessing elements with pointer notation:\n");

    for (int i = 0; i < size; i++) {
        // *(arr + i) is exactly the same as arr[i]
        // it just adds the index offset to the base memory address
        printf("element %d: %d\n", i, *(arr + i));
    }

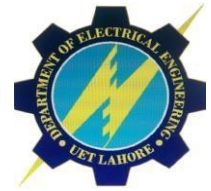
    return 0;
}
```

Output:

```
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ make task2
gcc -c task2.c
gcc -o task2 task2.o
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ ./task2
accessing elements with pointer notation:
element 0: 5
element 1: 15
element 2: 25
element 3: 35
element 4: 45
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ |
```



Programming Fundamentals Lab Manual



Task 3

Implement an array of pointers to store and display names of students.

Code:

```
#include <stdio.h>

int main() {
    // array of char pointers, basically just a list of strings
    const char *students[] = {"Umar", "Faizan", "Haseeb"};
    int count = sizeof(students) / sizeof(students[0]);
    printf("student list:\n");
    // looping through our array of pointers
    for (int i = 0; i < count; i++) {
        printf("%d. %s\n", i + 1, students[i]);
    }

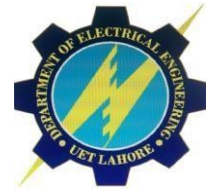
    return 0;
}
```

Output:

```
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ ./task3
student list:
1. Umar
2. Faizan
3. Haseeb
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ |
```



Programming Fundamentals Lab Manual



Bugged Code

```
#include <stdio.h>

void displayMessage()
{
    printf("This is a callback function\n")
}

void executeCallback(void callback())
{
    printf("Executing callback...\n");
    callback;
}

int main()
{
    executeCallback(displayMessage);
    return;
}
```

Corrected Code

```
#include <stdio.h>

void displayMessage() {
    // added the missing semicolon here
    printf("This is a callback function\n");
}

// fixed the parameter to be a proper function pointer format
void executeCallback(void (*callback)()) {
    printf("Executing callback...\n");

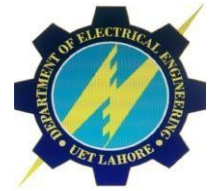
    // actually call the function instead of just referencing it
    callback();
}

int main() {
    executeCallback(displayMessage);

    // return needs a 0 since main returns an int
    return 0;
}
```



Programming Fundamentals Lab Manual



Output:

```
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ gted task4.c
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ make task4
gcc -c task4.c
gcc -o task4 task4.o
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$ ./task4
Executing callback...
This is a callback function
umar@umarr:~/Downloads/PF-Labs-main/Lab12_2025_EE_016$
```